



Roteiro de apresentação

CA-CFAR 2D: detecção adaptativa de alvos

Nicolas Marques
Ryan Alencar

Tempo estimado: 6 a 7 minutos
Falas separadas por responsabilidade e por slide



Slide 1 — CA-CFAR 2D

Boa tarde. Nosso trabalho apresenta uma implementação do algoritmo CA-CFAR em duas dimensões, aplicada a mapas Range-Doppler de radar. Além de implementar o algoritmo em C, buscamos demonstrar que o resultado é confiável usando uma estratégia de validação gratuita e reproduzível.

Ação: trocar para o slide 2.

Slide 2 — Detecção adaptativa em mapas Range-Doppler

Um radar produz um mapa de potência. Cada posição desse mapa representa uma combinação de distância e velocidade. O objetivo é identificar as células que podem representar alvos.

Um limiar fixo para o mapa inteiro pode funcionar mal quando o nível de ruído muda de uma região para outra. O CA-CFAR resolve esse problema calculando um limiar local. Ele observa a vizinhança de cada posição e procura valores que realmente se destacam daquele fundo.

A saída é uma nova matriz com zeros e uns. O valor um indica uma detecção. No gráfico, vemos as cinco posições detectadas no cenário Radar 05.

Ação: trocar para o slide 3.

Slide 3 — O que o CA-CFAR faz

Para cada célula analisada, chamada de CUT, usamos uma janela de nove por nove. A região central contém a CUT e as células de guarda. Essa região não entra na estimativa do ruído porque pode conter energia do próprio alvo.

As 72 células restantes formam a região de treinamento. Primeiro calculamos a média dessas células. Depois multiplicamos essa média pelo fator alfa, que neste projeto vale seis. Esse resultado forma o limiar local.

A detecção acontece somente quando a potência da CUT é maior que o limiar. A comparação é estrita. Se os valores forem iguais, não existe detecção.

Ação: trocar para o slide 4.

Slide 4 — Estruturas de dados e fluxo

A entrada contém 900 números, organizados em uma matriz de 30 por 30 valores do tipo double. A função `cfar_2d` percorre a região em que a janela completa cabe e grava as decisões em outra matriz de 30 por 30 inteiros.

O programa usa matrizes fixas e não precisa de alocação dinâmica. Isso deixa o fluxo simples e facilita uma futura adaptação para sistemas embarcados.

Como a janela precisa de quatro posições de margem, as quatro células de cada borda permanecem sem processamento.

Ação: trocar para o slide 5.

Slide 5 — Primeira dificuldade técnica

A primeira dificuldade foi definir corretamente os índices e o comportamento das bordas. Um erro de uma posição já mudaria as células usadas na estimativa do ruído. Para contornar isso, fixamos a região válida entre os índices quatro e vinte e cinco e criamos testes específicos para os limites.

O aprendizado foi que o tratamento de borda não é apenas um detalhe da implementação. Ele faz parte do comportamento esperado do algoritmo.

Transição para Ryan

Depois de definir o comportamento do algoritmo, ainda precisávamos responder uma pergunta importante: como provar que a implementação estava correta sem validar o código usando apenas ele mesmo? O Ryan vai explicar como fizemos essa validação e quais resultados obtivemos.

Slide 5 — Demais dificuldades técnicas

A segunda dificuldade foi justamente criar uma referência independente. Se o teste repetir a mesma lógica do programa, ele pode repetir também o mesmo erro. Por isso criamos um oráculo simples em NumPy e também comparamos o resultado com o NRL Tracker, uma biblioteca externa de referência.

Outra dificuldade apareceu no teste de falso alarme. Células vizinhas compartilham parte da janela de treinamento, então seus resultados não são estatisticamente independentes. Para evitar uma conclusão incorreta, selecionamos nove janelas de nove por nove que não se sobrepõem.

O principal aprendizado foi que a credibilidade depende tanto do desenho do experimento quanto do cálculo realizado pelo algoritmo.

Ação: trocar para o slide 6.

Slide 6 — Procedimento de teste

O procedimento usa as mesmas entradas em três caminhos. O primeiro executa a implementação C, que é o código sob teste. O segundo executa o oráculo NumPy, escrito diretamente a partir da definição matemática. O terceiro executa o CA-CFAR da biblioteca NRL Tracker, na versão 1.19.0.

As três implementações precisam produzir exatamente as mesmas detecções. O NumPy e o NRL também precisam concordar nos valores de ruído e limiar, com uma tolerância de dez elevado a menos doze.

Além dessa comparação, usamos casos analíticos, vinte mapas aleatórios com semente fixa e os cinco cenários do projeto. A compilação usa avisos estritos, e os sanitizers verificam memória e comportamento indefinido. O GitHub Actions repete a sequência completa automaticamente.

Ação: trocar para o slide 7.

Slide 7 — Resultados nos cenários versionados

No total, foram aprovados sete testes em C, dezessete testes determinísticos em Python e dois testes estatísticos. Os sanitizers não encontraram erros.

Nos cenários Radar 01, 02 e 05, todos os alvos foram detectados nas coordenadas esperadas. O Radar 03 não possui alvos e não apresentou detecções.

O Radar 04 contém um alvo de potência três, que ficou abaixo do limiar para alfa igual a seis. Esse resultado registra uma limitação conhecida da configuração atual e não uma falha da suíte de testes.

Ação: trocar para o slide 8.

Slide 8 — Pfa e Pd

Para avaliar a probabilidade de falso alarme, geramos cinco mil matrizes de ruído exponencial. As nove janelas independentes de cada matriz produziram 45 mil ensaios.

O Pfa teórico foi aproximadamente 0,003141. O valor observado foi 0,002978 e ficou dentro do intervalo de Wilson de 99 por cento. Portanto, o comportamento observado ficou compatível com a previsão matemática do CA-CFAR.

Também medimos a probabilidade de detecção para diferentes relações sinal-ruído. Ela foi 0,6 por cento em zero decibel, 6,3 por cento em cinco decibéis e chegou a 100 por cento a partir de dez decibéis.

Com esses resultados, concluímos que a implementação C está coerente com os dois referenciais e com o modelo estatístico testado. A validação ainda não cobre clutter não homogêneo, modelos Swerling ou desempenho em hardware embarcado. Esses pontos ficam como possibilidades de continuação do trabalho.

Obrigado.

Versão curta para controle de tempo

Se o tempo estiver terminando, usem estes cortes sem perder a conclusão principal:

- Nicolas: no slide 3, explique apenas que as 72 células estimam o ruído e que a CUT precisa superar seis vezes essa média.
- Ryan: no slide 6, cite os três referenciais e diga que as detecções foram idênticas, sem detalhar a tolerância.
- Ryan: no slide 7, apresente apenas o total de testes e a observação sobre o Radar 04.
- Ryan: no slide 8, informe que o Pfa teórico ficou dentro do intervalo de 99% e que a Pd chegou a 100% a partir de 10 dB.

Pronúncia sugerida

- CA-CFAR: “cá cé-far”.
- CUT: “cât” ou “célula sob teste”.
- Pfa: “pê-éfe-á”, probabilidade de falso alarme.
- Pd: “pê-dê”, probabilidade de detecção.
- NRL: “ene-erre-ele”.
- NumPy: “nâm-pai”.